

Anisa Aulia Alhaqi/18224080

function bSearch(p: BinTree, x: ElType) → boolean

ALGORITMA

```
    if(p = NIL) then
        → false
    if(p^.info = x) then
        → true
    else
        if(x < p^.info) then
            bSearch(p^.left, x)
        if(x > p^.info) then
            bSearch(p^.right, x)
```

function isBTree(p: BinTree) → boolean

KAMUS LOKAL

hl, hr, nl, nr, selisihH, selisihN: integer

ALGORITMA

```
    if(p = NIL) then
        → true
    hl ← height(p^.left)
    hr ← height(p^.right)
    nl ← nodeCount(p^.left)
    nr ← nodeCount(p^.right)
    selisihH ← absolute(hl-hr)
    selisihN ← absolute(nl-nr)
```

```
    → (selisihH ≤ 1) and (selisihN ≤ 1) and (isBTree(p^.left)) and
(isBTree(p^.right))
```

function height(p: BinTree) → integer

KAMUS LOKAL

hl, hr: integer

ALGORITMA

```
    if(p = NIL) then
        → 0
    else
        hl ← height(p^.left)
```

```

    hr ← height(p^.right)
    if(hl > hr) then
        → hl+1
    else
        → hr+1

```

```

function nodeCount(p: BinTree) → integer

```

ALGORITMA

```

    if(p = NIL) then
        → 0
    else
        → 1 + nodeCount(p^.left) + nodeCount(p^.right)

```

```

function buildBST(infos: array[0..99] of integer, neff: integer) →
BinTree

```

KAMUS LOKAL

```

    i: integer
    temp: ElType
    root: BinTree

```

ALGORITMA

```

    root ← NIL
    i traversal[0..neff-1]
        temp^.key ← infos[i]
        temp^.count ← 1
        inSearchTree(temp, root)
    → root

```